

MD005 – Knowledge Based Systems

Multi Layer Perceptron

Convolutional and Recurrent Neural Networks



Jordina Torrents Barrena

5th / 9th February 2026

Outline

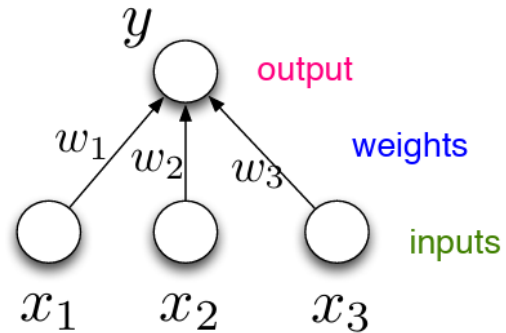
- Multi Layer Perceptron
 - Backpropagation
- Convolutional Neural Networks
 - Components
 - Architectures
 - Pretrained Models
 - Data Augmentation
- Introduction to Recurrent Neural Networks

Multi Layer Perceptron



Overview

Recall the simple neural – like unit:



$$y = g \left(b + \sum_i x_i w_i \right)$$

The equation is annotated with colored arrows: a pink arrow points to y labeled "output"; a blue arrow points to b labeled "bias"; a red arrow points to g labeled "nonlinearity"; a green arrow points to x_i labeled "i'th input"; and a blue arrow points to w_i labeled "i'th weight".

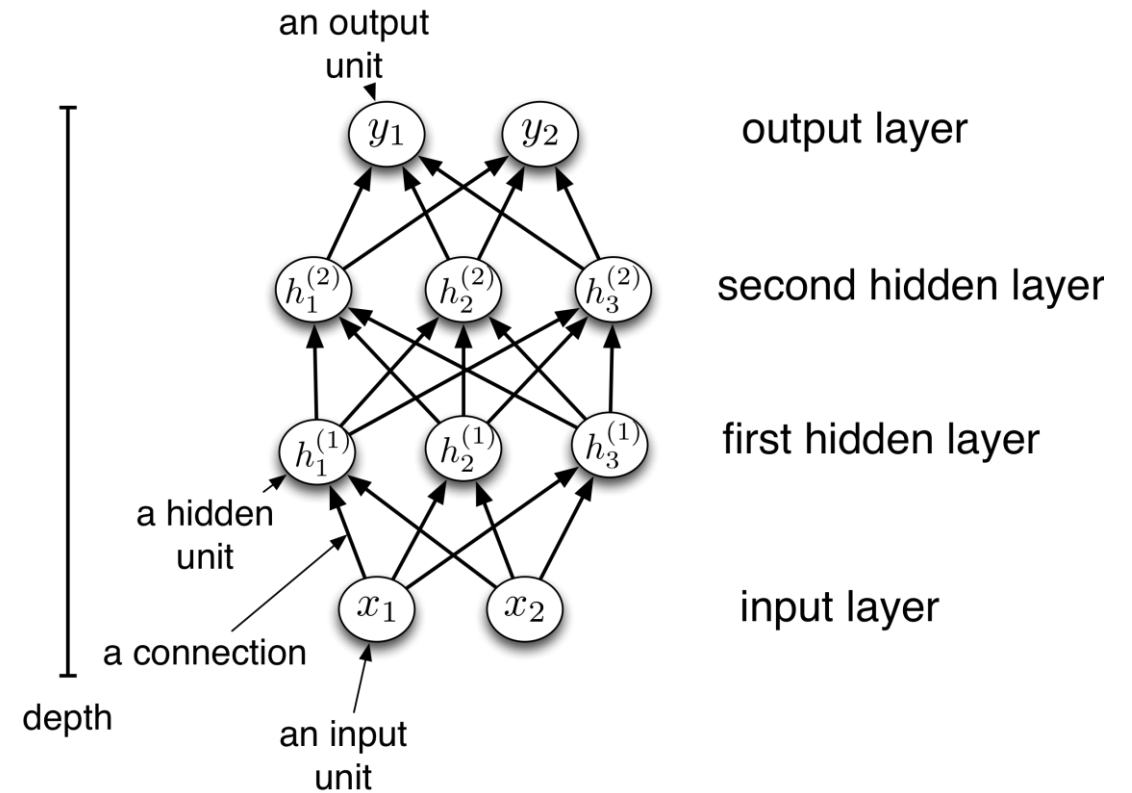
These units are much more powerful if we connect many of them into a neural network.

Design choices so far:

- Task: regression, binary classification, multiway classification.
- Model/Architecture: linear, log-linear, feed-forward neural network.
- Loss function: squared error, 0–1 loss, cross-entropy, hinge loss.
- Optimization algorithm: direct solution, gradient descent, perceptron.

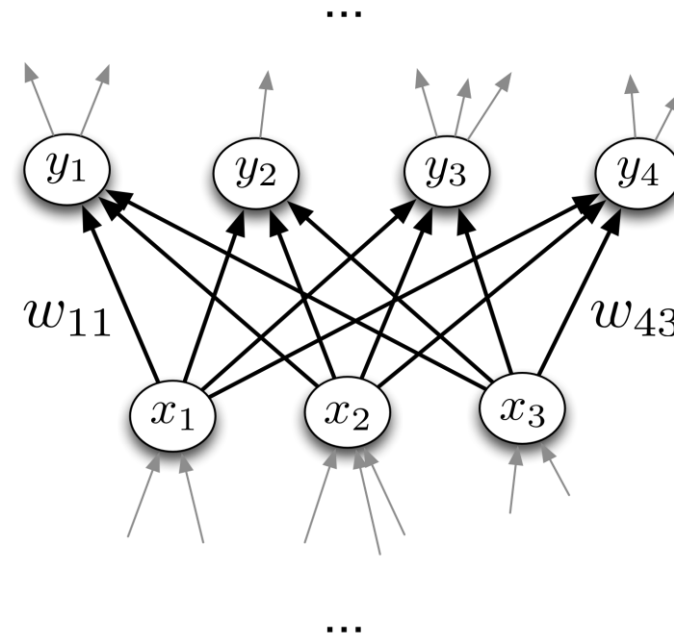
Multi Layer Perceptions

- We can connect lots of units together into a directed acyclic graph.
- This gives a feed-forward neural network. That's in contrast to recurrent neural networks, which can have cycles. (*we'll talk about those later*).
- Typically, units are grouped together into layers.



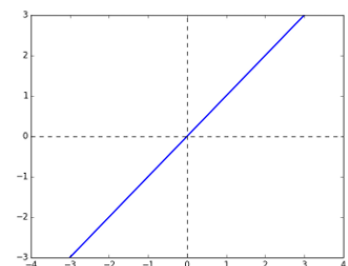
Multi Layer Perceptions

- Each layer connects N input units to M output units.
 - In the simplest case, all input units are connected to all output units. We call this a fully connected layer (*we'll consider other layer types later*).
 - Note: the inputs and outputs for a layer are distinct from the inputs and outputs to the network.
-
- Recall from multiway logistic regression: this means we need an $M \times N$ weight matrix.
 - The output units are a function of the input units:
$$\mathbf{y} = \mathbf{f}(\mathbf{x}) = \phi(\mathbf{W}\mathbf{x} + \mathbf{b})$$
 - A multilayer network consisting of fully connected layers is called a multilayer perceptron. Despite the name, it has nothing to do with perceptrons!



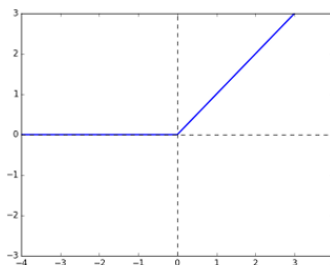
Multi Layer Perceptions

Some activation functions:



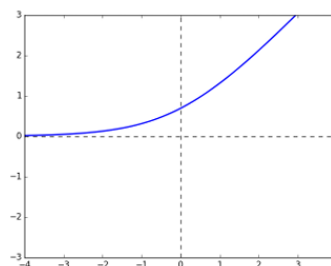
Linear

$$y = z$$



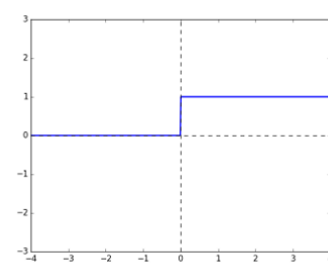
Rectified Linear Unit (ReLU)

$$y = \max(0, z)$$



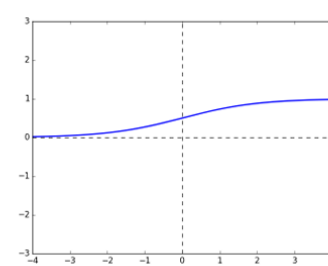
Soft ReLU

$$y = \log 1 + e^z$$



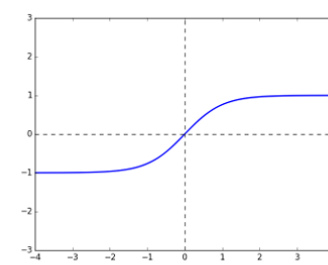
Hard Threshold

$$y = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$



Logistic

$$y = \frac{1}{1 + e^{-z}}$$



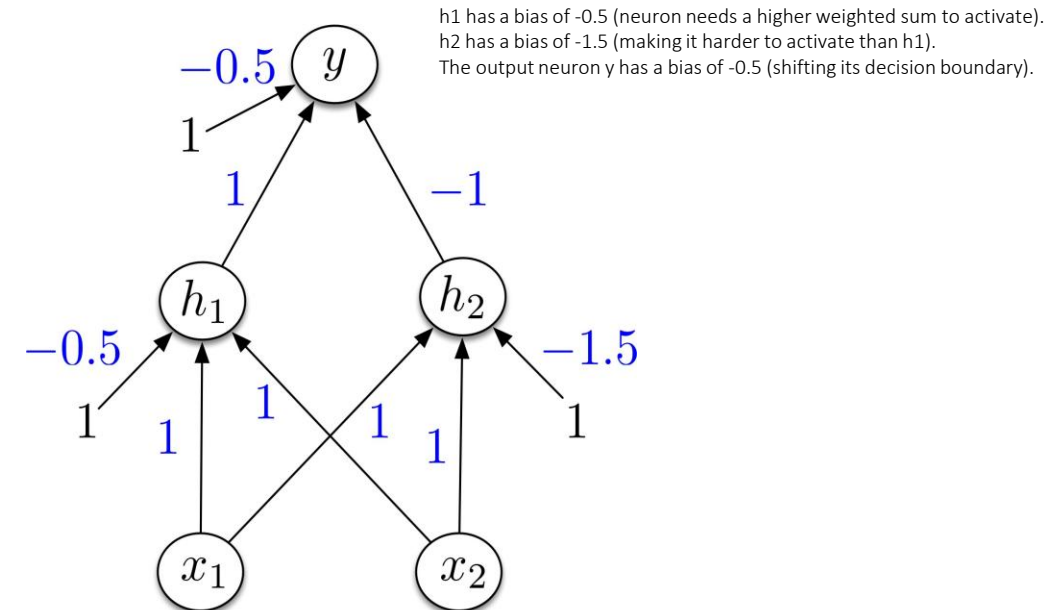
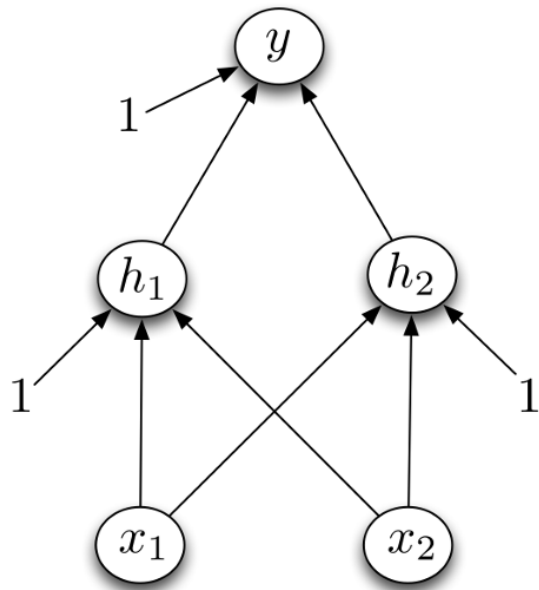
Hyperbolic Tangent (tanh)

$$y = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- **Linear:** Outputs a weighted sum of inputs without non-linearity, useful in regression but not deep networks.
- **ReLU:** Outputs zero for negative inputs and the input itself for positive values, preventing vanishing gradients.
- **Soft ReLU:** A smooth approximation of ReLU, thus ensuring differentiability.
- **Hard Threshold:** Outputs 0 for negative inputs and 1 for positive inputs, acting as a step function.
- **Logistic (Sigmoid):** Maps inputs to (0,1) range, often used for probability estimation but suffers from vanishing gradients.
- **Hyperbolic Tangent (Tanh):** Maps inputs to (-1,1) range, offering better symmetry around zero compared to sigmoid.

Multi Layer Perceptions

Assume Hard Threshold activation function:



Left: represents a simple MLP structure, where inputs (x_1, x_2) are passed through hidden neurons (h_1, h_2) and then to the output neuron (y).

Right: introduces specific weights and biases (in blue), showing how each connection contributes to the computation.

Each neuron applies a weighted sum of its inputs, and the Hard Threshold activates the neuron if the sum exceeds a certain threshold, otherwise zero.
(The diagram demonstrates how different weight assignments influence neuron activation and final output)

Multi Layer Perceptions

Each layer computes a function, so the network computes a composition of functions:

$$h(1) = f(1)(x)$$

$$h(2) = f(2)(h(1))$$

▪

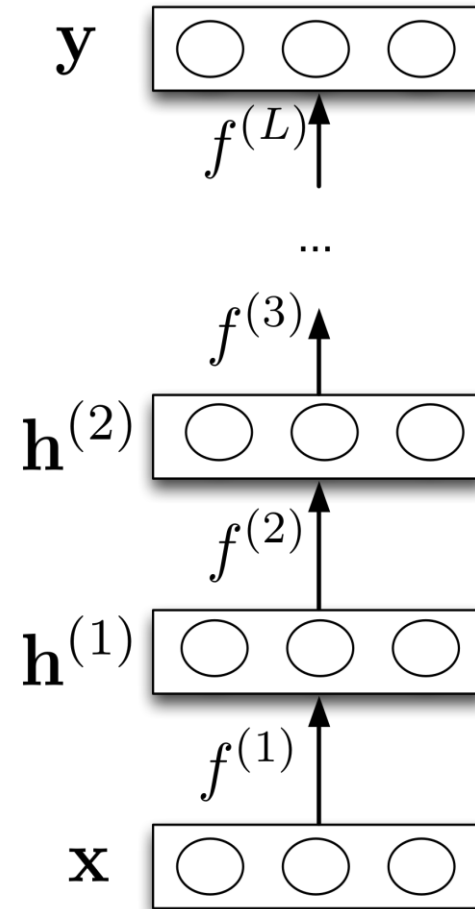
▪

$$y = f(L)(h(L-1))$$

Or more simply:

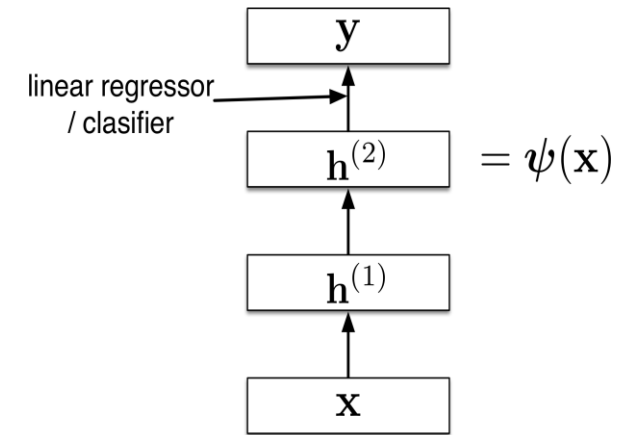
$$y = f(L) \circ \dots \circ f(1)(x)$$

Neural networks provide **modularity**: we can implement each layer's computations as a black box.

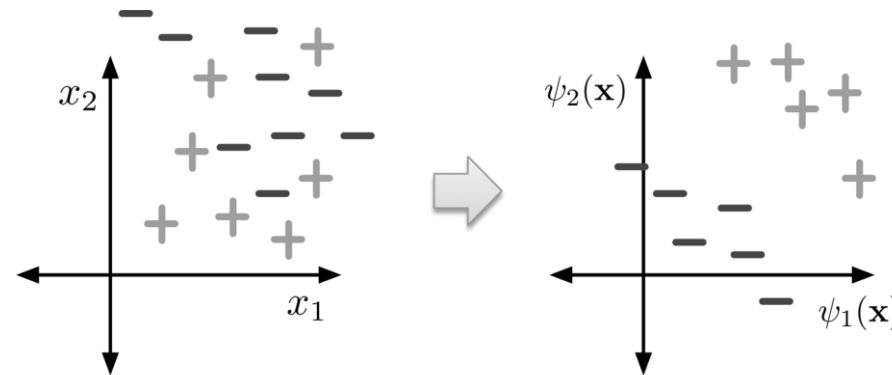


Feature Learning

- **Neural networks** can be viewed as a **way of learning features**, since they learn features automatically by **extracting patterns from raw data through multiple layers**, **eliminating** the need for **manual feature engineering**.
- **Early layers capture simple features** (e.g., edges in images), while **deeper layers learn complex representations** (e.g., shapes, objects).
- This **hierarchical learning enables** neural networks to **adapt to various tasks** like image recognition, speech processing, and natural language understanding by **optimizing features directly from data during training**.



The goal (*learn patterns and representations from data to make accurate predictions or decisions*):



Feature Learning

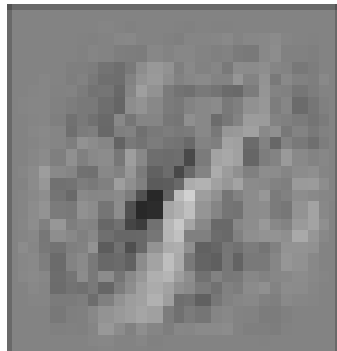
Input representation of a digit: 784-dimensional vector

[illegible]

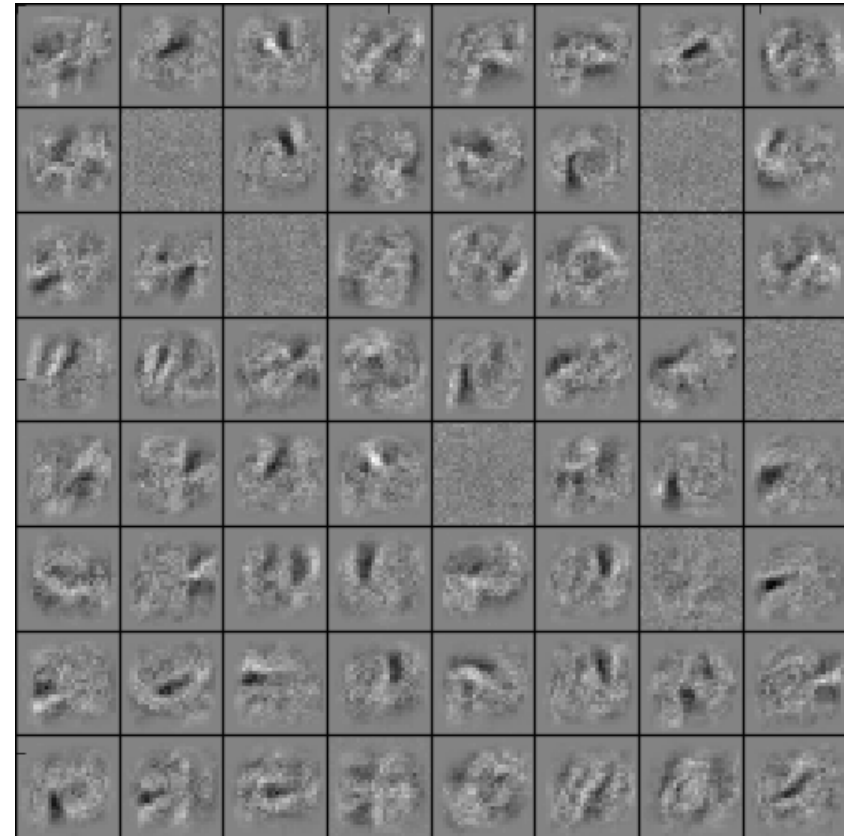
How handwritten digits are represented as feature vectors. The **784-dimensional vector** comes from a **28×28 grayscale image**, where **each pixel's intensity (ranging from 0 to 255)** is flattened into a single row of numbers. The displayed matrix visually represents these pixel values, forming the shape of a digit. This numerical representation is crucial for feature learning, allowing models to detect patterns and classify digits effectively.

Feature Learning

- Each first-layer hidden unit computes $\sigma(\mathbf{w}_i^T \mathbf{x})$
- Here is one of the **weight vectors** (also called a **feature**). It's reshaped into an image, with **gray = 0**, **white = +**, **black = -**.
- To compute $\mathbf{w}_i^T \mathbf{x}$, multiply the corresponding pixels, and sum the result.



There are **256 first-level features** total:



MLP Training Algorithm

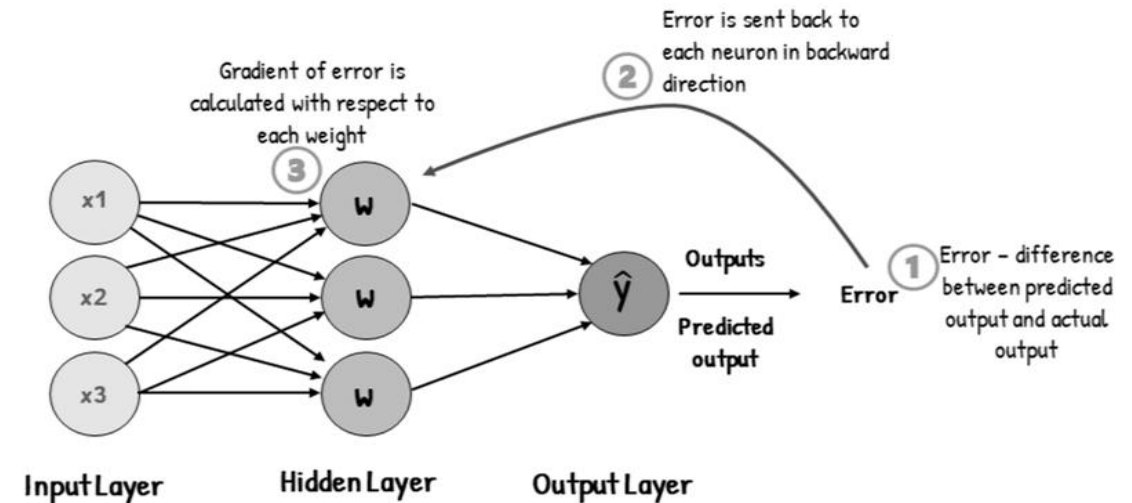
A MLP neural network **trained** using the **Backpropagation learning algorithm** is one of the most powerful forms of supervised neural network system.

The training of such a network involves **three stages**:

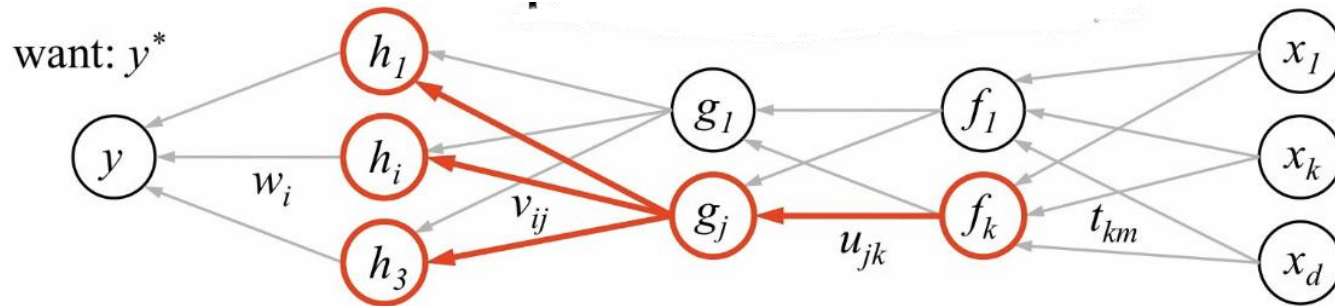
- **feedforward** of the input training pattern
- **calculation and backpropagation** of the associated **error**
- **adjustment of the weights**

This procedure is repeated for each pattern over several complete passes (epochs) through the training set.

After training, application of the net only involves the computations of the feedforward phase.



Backpropagation



1. receive new observation $\mathbf{x} = [x_1 \dots x_d]$ and target y^*
2. **feed forward:** for each unit g_j in each layer $1 \dots L$
compute g_j based on units f_k from previous layer: $g_j = \sigma \left(u_{j0} + \sum_k u_{jk} f_k \right)$
3. get prediction y and error $(y - y^*)$
4. **back-propagate error:** for each unit g_j in each layer $L \dots 1$

(a) compute error on g_j

$$\underbrace{\frac{\partial E}{\partial g_j}}_{\text{should } g_j \text{ be higher or lower?}} = \sum_i \underbrace{\sigma'(h_i)}_{\text{how } h_i \text{ will change as } g_j \text{ changes}} \underbrace{v_{ij} \frac{\partial E}{\partial h_i}}_{\text{was } h_i \text{ too high or too low?}}$$

(b) for each u_{jk} that affects g_j

(i) compute error on u_{jk}

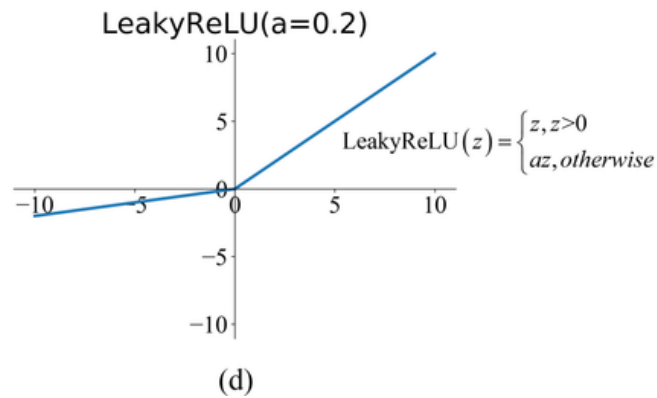
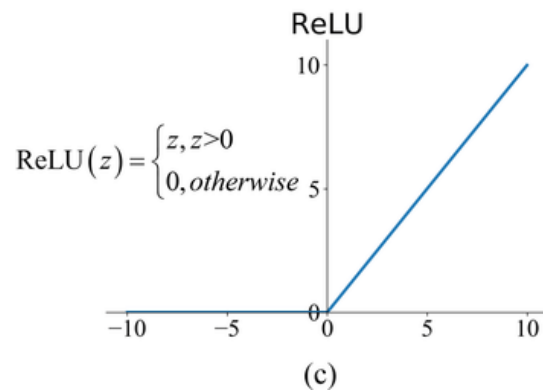
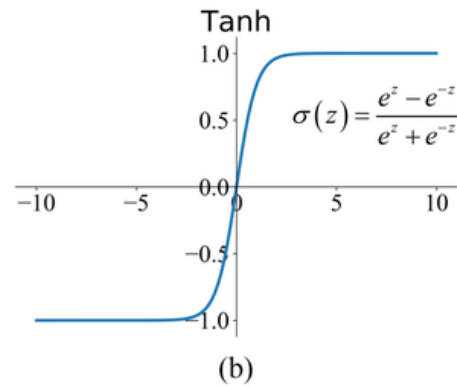
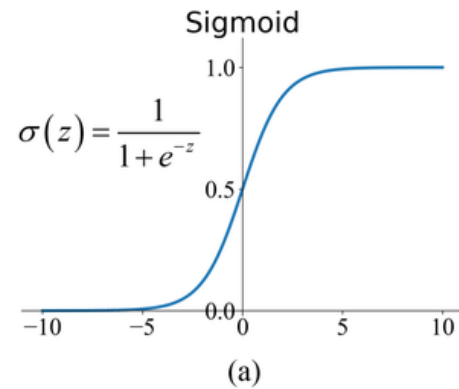
$$\frac{\partial E}{\partial u_{jk}} = \underbrace{\frac{\partial E}{\partial g_j}}_{\text{do we want } g_j \text{ to be higher/lower}} \underbrace{\sigma'(g_j) f_k}_{\text{how } g_j \text{ will change if } u_{jk} \text{ is higher/lower}}$$

(ii) update the weight

$$u_{jk} \leftarrow u_{jk} - \eta \frac{\partial E}{\partial u_{jk}}$$

- **Backpropagation** is an iterative optimization method used to train multi-layer neural networks by minimizing the error between predicted and actual outputs.
- In the **forward pass**, the input data is passed through the network, layer by layer, with each neuron applying a weighted sum of inputs followed by a non-linear activation function. The final layer produces an output, and the loss function quantifies the error.
- In the **backward pass**, this error is propagated backward using the chain rule of differentiation to compute gradients of the loss with respect to each weight. These gradients indicate the direction in which each weight should be updated to reduce the error. Using **gradient descent** (or its variants), the weights are adjusted accordingly.
- This process repeats iteratively over multiple training samples until the network converges to an optimal set of parameters, effectively learning to map inputs to outputs.

Activation Functions



- **Sigmoid:** This activation function **compresses inputs into a range between 0 and 1**, making it useful for probability-based outputs. However, it has the downside of **causing vanishing gradients**, which **slows down training** in deep networks.
- **ReLU (Rectified Linear Unit):** It **outputs the input directly if it's positive but sets negative inputs to zero**. This makes it **computationally efficient and speeds up learning**, but it **can lead to dead neurons**, where some neurons **stop learning** if they always receive negative values.
- **Leaky ReLU:** A modified version of ReLU that **allows a small non-zero output for negative inputs instead of setting them to zero**. This helps **prevent dead neurons** and allows gradients to flow even for negative values.
- **Tanh (Hyperbolic Tangent):** **Similar to the sigmoid function but ranges between -1 and 1, making it zero-centered**. This helps in training by **keeping activations balanced**, but it **still suffers from the vanishing gradient problem** in deep networks.

Optimizers

An optimizer is an **algorithm that adjusts the model's weights and biases to minimize the loss function during training**. It updates these parameters based on the gradients computed through backpropagation, guiding the network toward better performance.

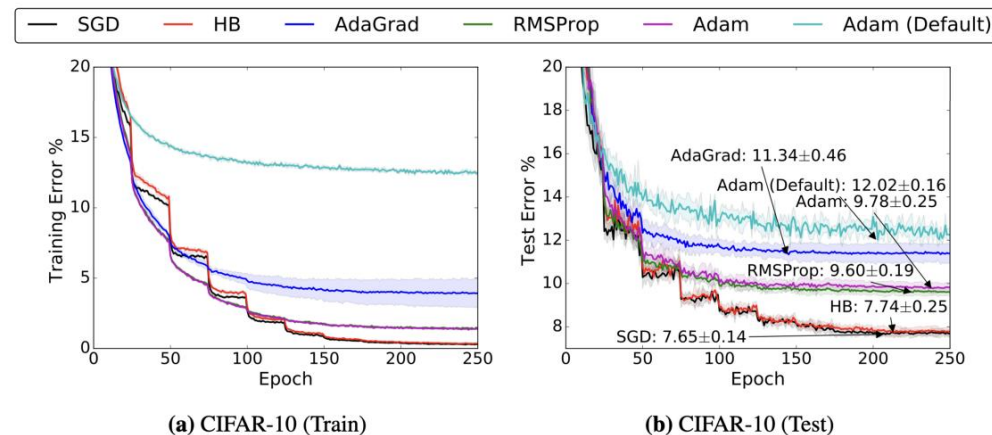
1. Gradient Descent-Based Optimizers:

- **Stochastic Gradient Descent (SGD)**: Updates weights using a small batch (or a single sample) at a time. Can have momentum to accelerate training.
- **Mini-batch SGD**: Uses a subset of data for each update, balancing efficiency and stability.
- **Momentum-based SGD**: Adds momentum to prevent oscillations and improve convergence.

2. Adaptive Learning Rate Optimizers:

- **Adam** (Adaptive Moment Estimation): Combines momentum and adaptive learning rates for efficient and stable updates.
- **Adagrad**: Adapts learning rates based on past gradients but can cause very small updates over time.
- **RMSprop**: Similar to Adagrad but uses an exponentially decaying average to prevent diminishing updates.
- **Adadelta**: A refined version of Adagrad that removes the need for manually setting a learning rate.

Each optimizer has strengths and weaknesses, with **SGD being simple and effective**, while **Adam is widely used due to its adaptability**.



Test Stopping Condition

For example, after each epoch of training the **Root Mean Square Error (RMSE)** of the network for all of the patterns in a separate validation set is calculated.

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

$\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n$ are predicted values

$y_1, y_2, \dots, y_n$ are observed values

n is the number of observations

Training is terminated when the RMSE value for the validation set either starts to increase or remains constant over several epochs.

This prevents the network from being **overtrained** (i.e. memorising the training set) and **ensures** that the **ability of the network to generalise** (i.e. correctly classify non-trained patterns) will be at its maximum.

Factors Affecting Network Performance

Number of hidden nodes: 1) too many and the network may memorise training set, 2) too few and the network may not learn the training set.

Initial weight set: 1) some starting weight sets may lead to a local minimum, 2) other starting weight sets avoid the local minimum.

Training set: 1) must be statistically relevant, 2) patterns should be presented in random order.

Data representation: 1) low level - very large training set might be required, 2) high level – human expertise required.

Expressive Power

We've seen that there are some functions that linear classifiers can't represent. **Are deep networks any better?**

Any **sequence of linear layers** can be equivalently represented with a **single linear layer**.

$$y = \underbrace{\mathbf{W}^{(3)}\mathbf{W}^{(2)}\mathbf{W}^{(1)}}_{\triangleq \mathbf{W}'} \mathbf{x}$$

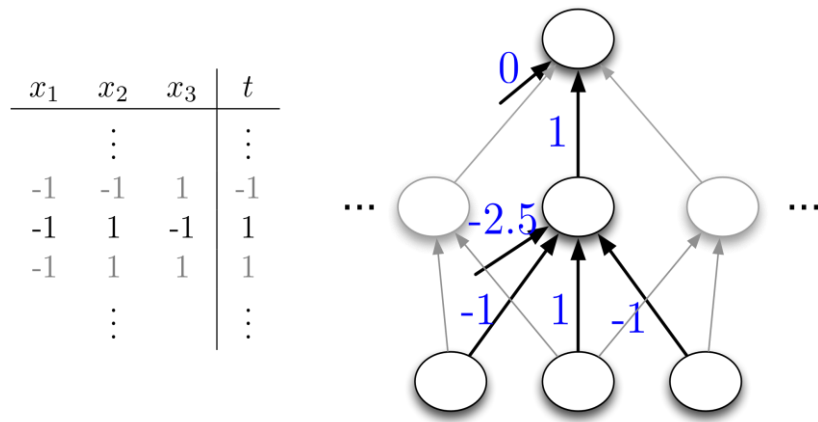
Deep linear networks are no more expressive than linear regression! Linear layers do have their uses — *stay tuned!*

- Multilayer feed-forward neural networks with nonlinear activation functions are **universal approximators**: they can approximate any function arbitrarily well.
- This has been shown for various activation functions (e.g., thresholds, logistic, ReLU, etc.)
 - Even though ReLU is “almost” linear, it's nonlinear enough!

Expressive Power

Universality for binary inputs and targets:

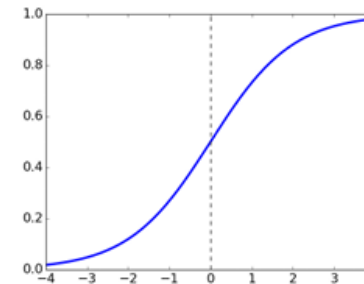
- Hard threshold hidden units, linear output.
- Strategy: 2^D hidden units, each of which responds to one particular input configuration.



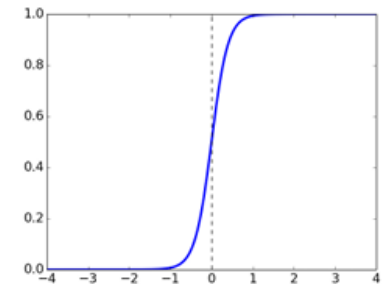
Only requires **one hidden layer**, though it needs to be extremely wide!

What about the logistic activation function?

- You can approximate a hard threshold by scaling up the weights and biases.
- This is good: logistic units are differentiable, so we can tune them with gradient descent.



$$y = \sigma(x)$$



$$y = \sigma(5x)$$

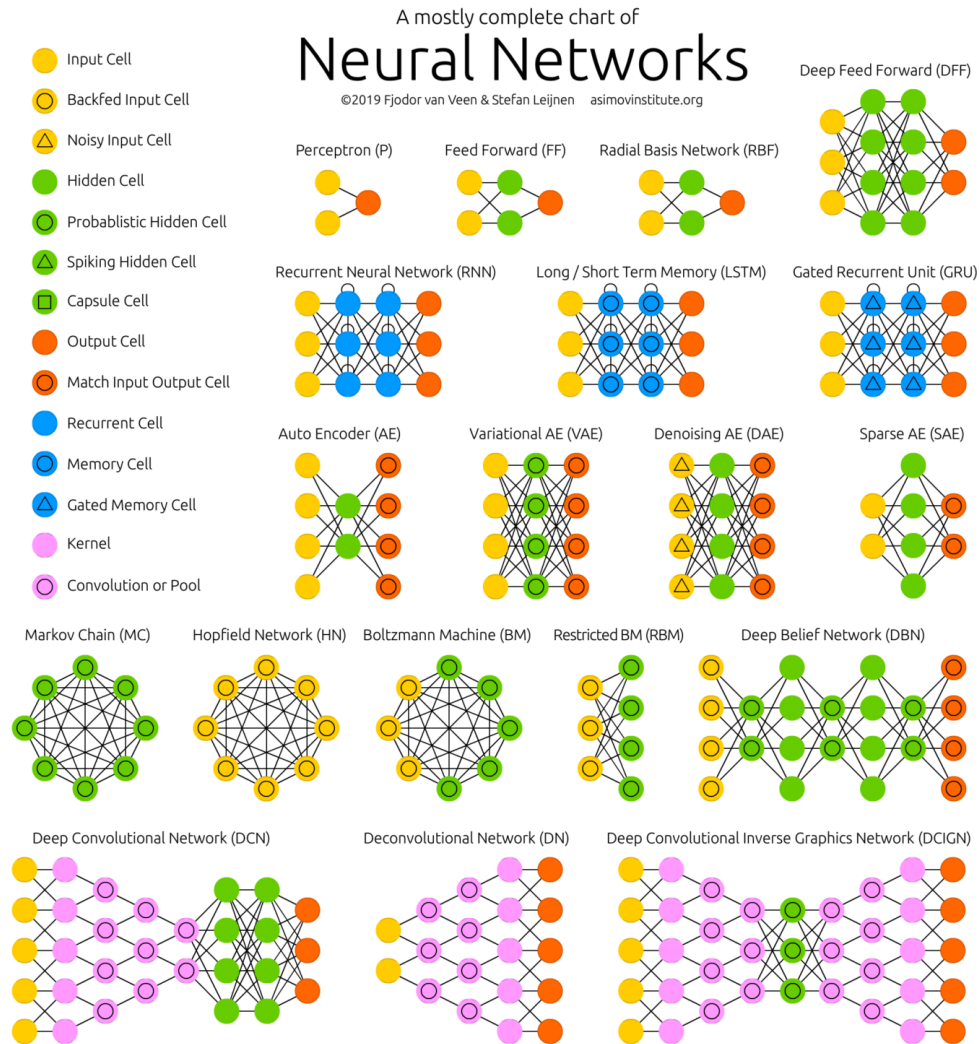
Expressive Power

The **limits of universality** in a MLP refer to the constraints and challenges of its ability to approximate functions, despite the universal approximation theorem stating that an MLP with at least one hidden layer can approximate any continuous function given enough neurons.

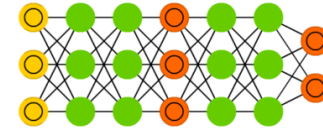
Key limits:

1. **Finite Resources:** In practice, an MLP cannot have infinite layers, limiting its ability to approximate highly complex functions.
2. **Training and Optimization:** Universal approximation assumes an optimal set of weights, but in reality, finding these weights requires efficient training algorithms, which can struggle with local minima, vanishing gradients, or slow convergence.
3. **Generalization vs. Overfitting:** While an MLP can approximate any function, it may overfit if trained on limited data or fail to generalize when applied to unseen inputs.
4. **Computational Constraints:** As the complexity of the function increases, an MLP may require an impractically large number of neurons or layers, making computation expensive.
5. **Expressiveness of Activation Functions:** Some activation functions (e.g., ReLU, sigmoid) impact the ability of the network to represent certain types of functions efficiently.

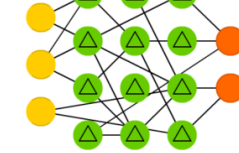
The Neural Network Zoo



Generative Adversarial Network (GAN)



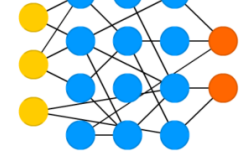
Liquid State Machine (LSM)



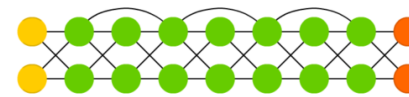
Extreme Learning Machine (ELM)



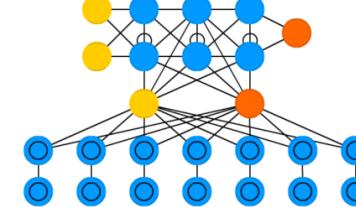
Echo State Network (ESN)



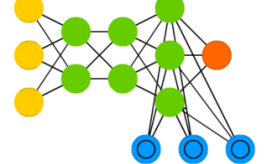
Deep Residual Network (DRN)



Differentiable Neural Computer (DNC)



Neural Turing Machine (NTM)



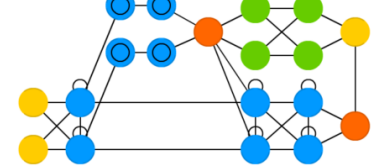
Capsule Network (CN)



Kohonen Network (KN)



Attention Network (AN)



Multi Layer Perceptron Working Lab



Examples To Play Around...

Examples to refresh how to play around with neural networks:

- Binary Classification (*binary_classification* Jupyter notebook)
 - `from sklearn.datasets import load_breast_cancer`
- Multiclass Classification (*multiclass_classification* Jupyter notebook)
 - `from sklearn.datasets import load_wine`

Key points to bear in mind:

For classification:

If binary:

- Single neuron
- Binary cross entropy

If multiclass:

- n outputs where n is the number of classes (if one-hot encoded)
- Cross entropy

For regression:

- Single neuron for single element regressor
- As many neurons as variables to predict
- Distance metrics: MSE/RMSE/MAE ...

The complex the problem, the more computation required (neurons), along with epochs and learning rate.

Convolutional Neural Networks



Motivation

Standard Dense Layer for an image input:

```
x = Input((640, 480, 3), dtype='float32')
# shape of x is: (None, 640, 480, 3)
x = Flatten()(x)
# shape of x is: (None, 640 x 480 x 3)
z = Dense(1000)(x)
```

How many **parameters** in the Dense layer?

$640 \times 480 \times 3 \times 1000$ (weights) + 1000 (biases) = 922M !!!

Spatial organization of the input is **destroyed** by Flatten.

We never use Dense layers directly on large images. Most standard solution is convolution layers.

Fully Connected Network: MLP

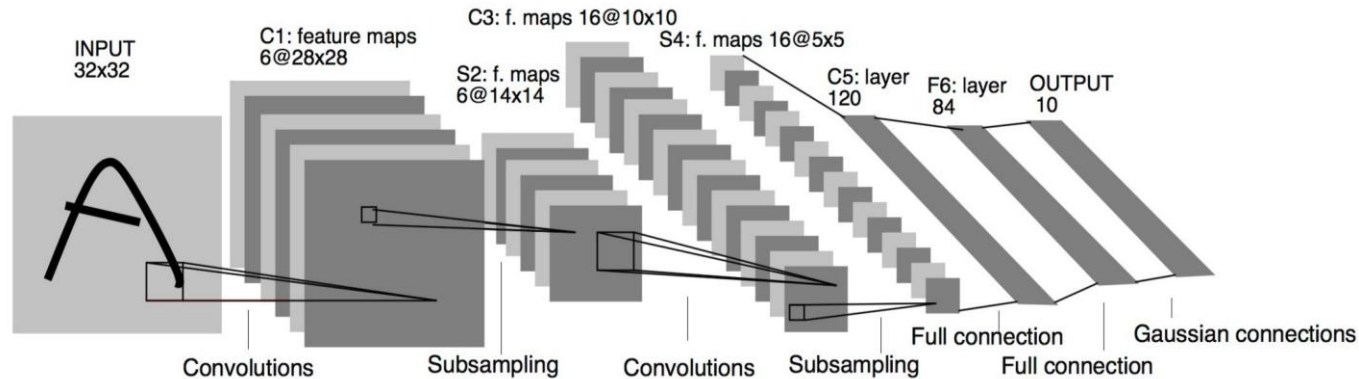
```
input_image = Input(shape=(28, 28, 1))
x = Flatten()(input_image)
x = Dense(256, activation='relu')(x)
x = Dense(10, activation='softmax')(x)
mlp = Model(inputs=input_image, outputs=x)
```

Convolutional Network

```
input_image = Input(shape=(28, 28, 1))
x = Conv2D(32, 5, activation='relu')(input_image)
x = MaxPool2D(2, strides=2)(x)
x = Conv2D(64, 3, activation='relu')(x)
x = MaxPool2D(2, strides=2)(x)
x = Flatten()(x)
x = Dense(256, activation='relu')(x)
x = Dense(10, activation='softmax')(x)
convnet = Model(inputs=input_image, outputs=x)
```

2D spatial organization of features preserved until *Flatten*.

Convolutional Networks



LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition.

Deep learning model designed for **processing grid-like data**, such as images, by **using convolutional layers** to automatically **extract hierarchical features**. They **leverage spatial hierarchies, weight sharing, and local receptive fields** to efficiently detect patterns like edges, textures, and objects, making them the foundation of modern computer vision.

- **Eyes (Retina) → Input Layer**: Captures raw visual information.
- **Early Visual Cortex (V1) → Initial Conv Layers**: Detects basic features (e.g., edges, textures).
- **Intermediate Visual Cortex (V2, V4) → Deeper Conv Layers**: Identifies complex patterns (e.g., shapes, colors).
- **Inferotemporal Cortex (IT) → Fully Connected Layers**: Recognizes high-level objects, categories, and meaning.
- **Decision Making (Prefrontal Cortex) → Output Layer**: Processes final classification (decision).

Local Connectivity

- A neuron depends only on a few local input neurons.
- Translation invariance.

Comparison to Fully Connected

- Parameter sharing: reduce overfitting.
- Make use of spatial structure: strong prior for vision!

Convolution

3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

A convolution is a **mathematical operation** where a **small filter** (kernel) **slides over an input** (e.g., an image) to detect patterns like edges or textures. It helps **extract local features** by **computing weighted sums**, **reducing parameters**, and **preserving spatial relationships**.

- x is a 3×3 chunk (dark area) of the image (*blue array*)
- Each output neuron is parametrized with the 3×3 weight matrix $\mathbf{w}$ (*small numbers*)

The activation obtained by sliding the 3×3 window and computing:

$$z(x) = \text{relu}(\mathbf{w}^T x + b)$$

Why Convolution?

Discrete convolution (actually cross-correlation) between two functions f and g :

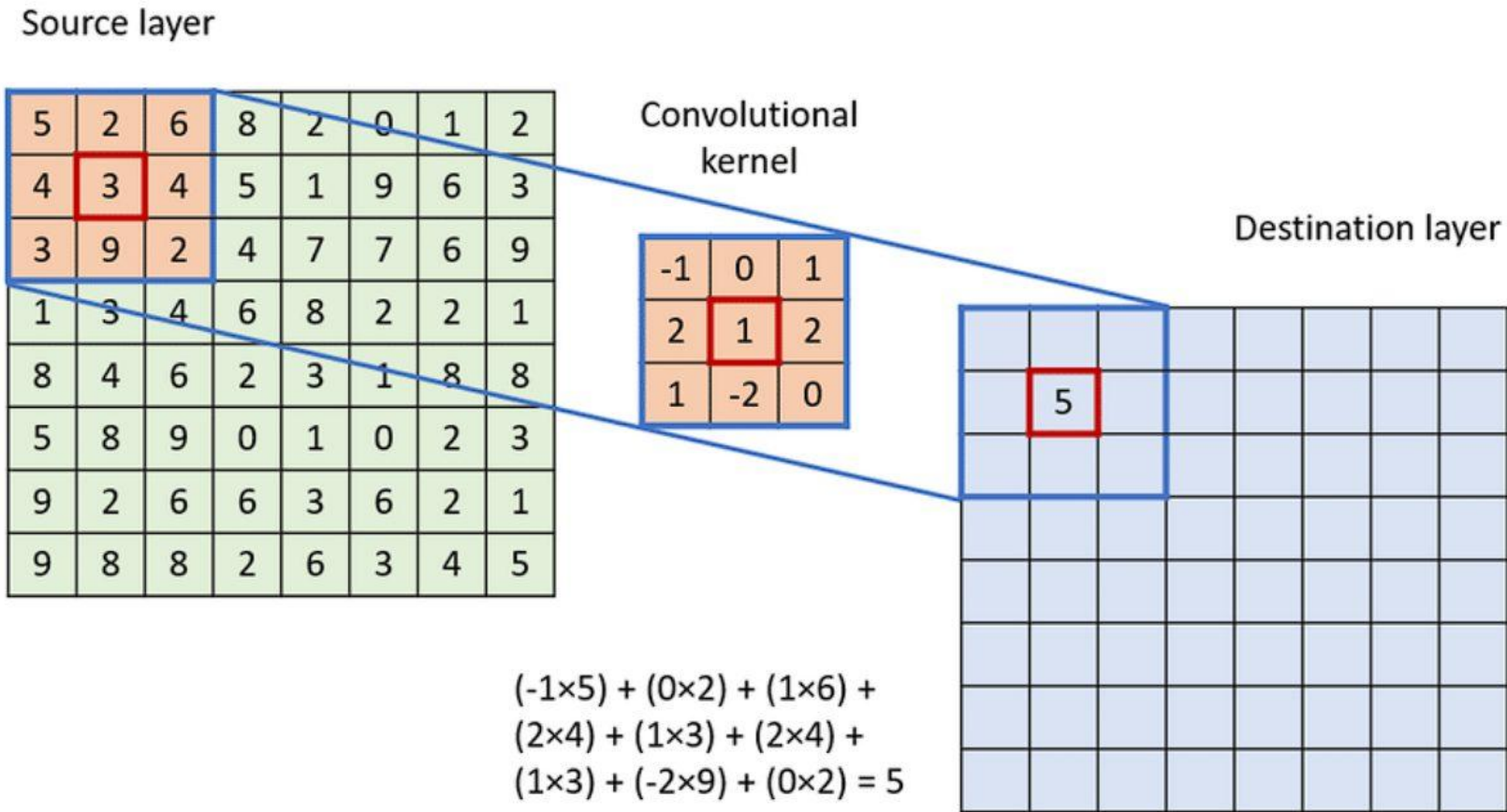
$$(f \star g)(x) = \sum_{a+b=x} f(a) \cdot g(b) = \sum_a f(a) \cdot g(x+a)$$

2D-convolutions (actually 2D cross-correlation):

$$(f \star g)(x, y) = \sum_n \sum_m f(n, m) \cdot g(x+n, y+m)$$

f is a convolution **kernel** or **filter** applied to the 2-d map g (our image)

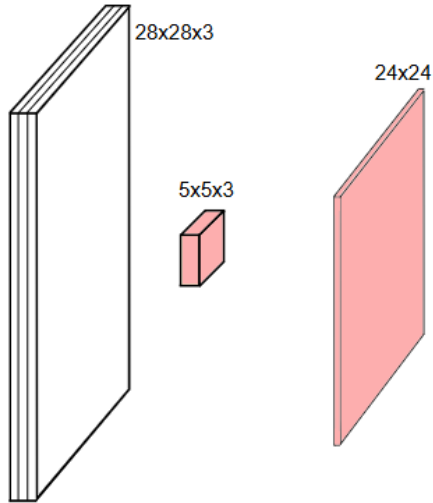
Example: Image Convolution



Channels

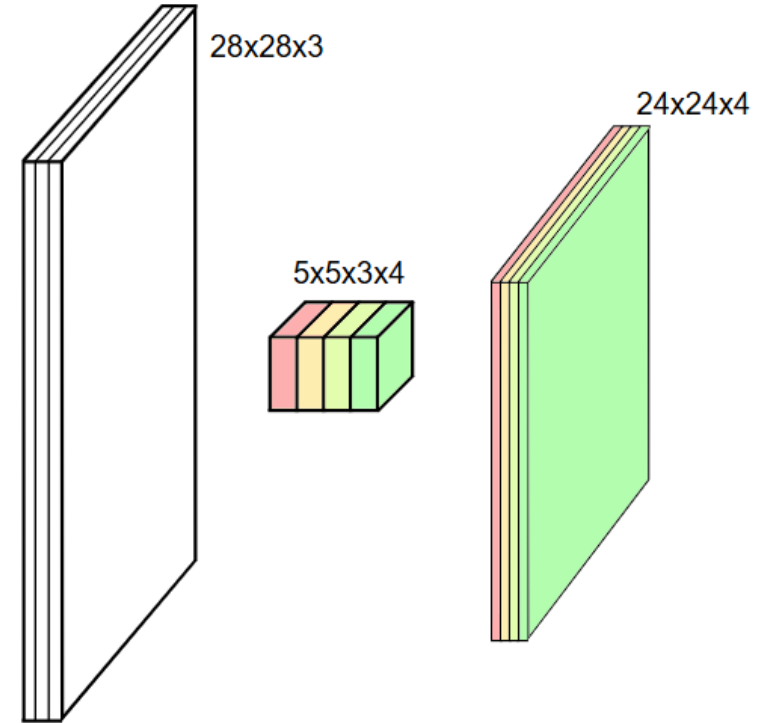
Colored image = tensor of shape (*height, width, channels*).

Convolutions are usually computed for each channel and summed:



$$(k \star im^{color}) = \sum_{c=0}^2 k^c \star im^c$$

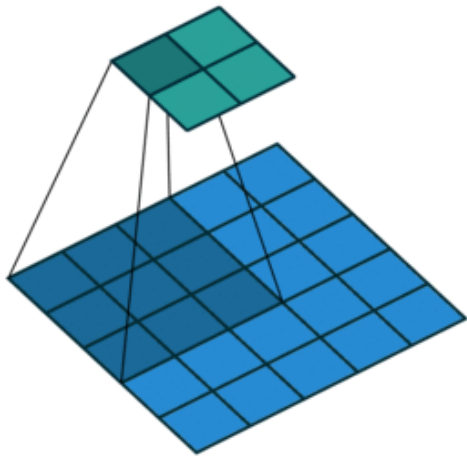
Multiple Convolutions



- Kernel size aka receptive field (usually 1, 3, 5, 7, 11).
- Output dimension: $length - kernel_size + 1$

Strides

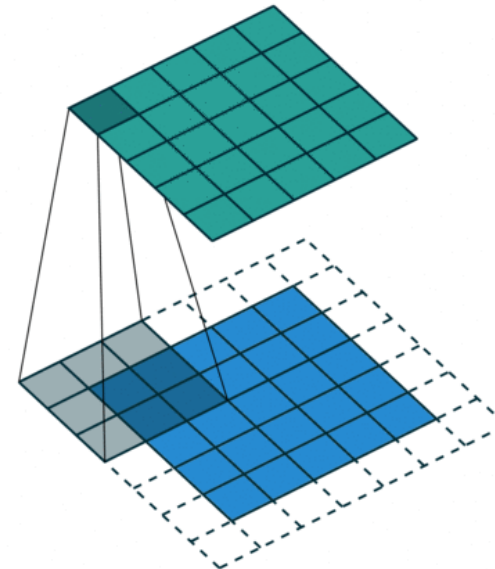
- Strides: increment step size for the convolution operator.
- Reduces the size of the output map.



Example with kernel size 3×3 and a stride of 2 (image in blue)

Padding

- Padding: artificially fill borders of image.
- Useful to keep spatial dimension constant across filters.
- Useful with strides and large receptive fields.
- Usually: fill with 0s.



Dealing with Shapes

Kernel or Filter shape (F, F, C^i, C^o)

- $F \times F$ kernel size,
- C^i input channels
- C^o output channels



Number of parameters: $(F \times F \times C^i + 1) \times C^o$

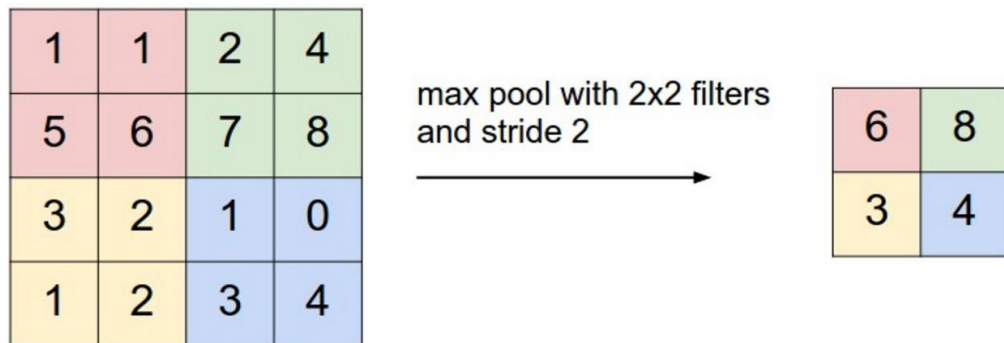
Activations or Feature maps shape:

- Input (W^i, H^i, C^i)
- Output (W^o, H^o, C^o)

$$W^o = (W^i - F + 2P)/S + 1$$

Pooling

- Pooling **reduces spatial dimensions** in CNNs **while retaining essential features**.
- Spatial dimension reduction.
- Local invariance.
- No parameters: max or average of 2x2 units.



Schematic from Stanford <http://cs231n.github.io/convolutional-networks>

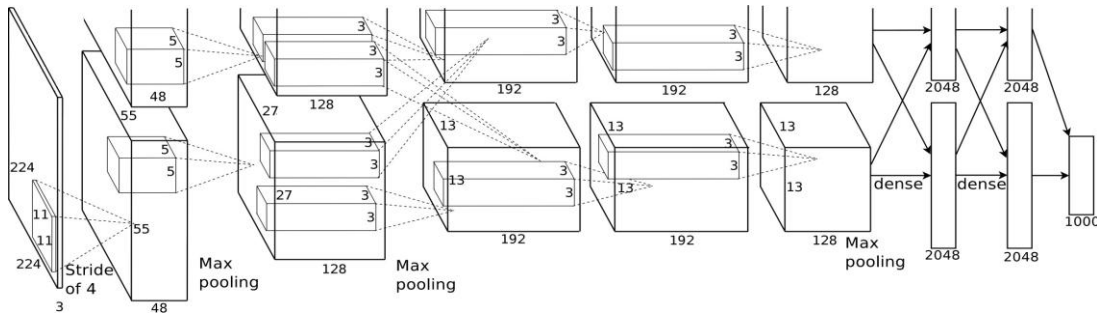
The **main types** are:

- **Max Pooling:** takes the maximum value in a region, highlighting dominant features.
- **Average Pooling:** computes the average value in a region, preserving smoother features.
- **Global Pooling:** reduces the entire feature map to a single value per channel (e.g., Global Average Pooling, GAP).
- **L2 Pooling:** uses the L2 norm in a region for a balance between max and average pooling.

Pooling **helps with dimensionality reduction, computational efficiency, and translation invariance.**

AlexNet

Deep convolutional neural network that **introduced ReLU activations, dropout, and overlapping max pooling**, significantly improving image classification performance. It has **eight layers** (five convolutional and three fully connected) and was a breakthrough in deep learning after **winning the 2012 ImageNet competition**.



INPUT:	[227x227x3]	
CONV1:	[55x55x96]	96 11x11 filters at stride 4, pad 0
MAX POOL1:	[27x27x96]	3x3 filters at stride 2
CONV2:	[27x27x256]	256 5x5 filters at stride 1, pad 2
MAX POOL2:	[13x13x256]	3x3 filters at stride 2
CONV3:	[13x13x384]	384 3x3 filters at stride 1, pad 1
CONV4:	[13x13x384]	384 3x3 filters at stride 1, pad 1
CONV5:	[13x13x256]	256 3x3 filters at stride 1, pad 1
MAX POOL3:	[6x6x256]	3x3 filters at stride 2
FC6:	[4096]	4096 neurons
FC7:	[4096]	4096 neurons
FC8:	[1000]	1000 neurons (softmax logits)

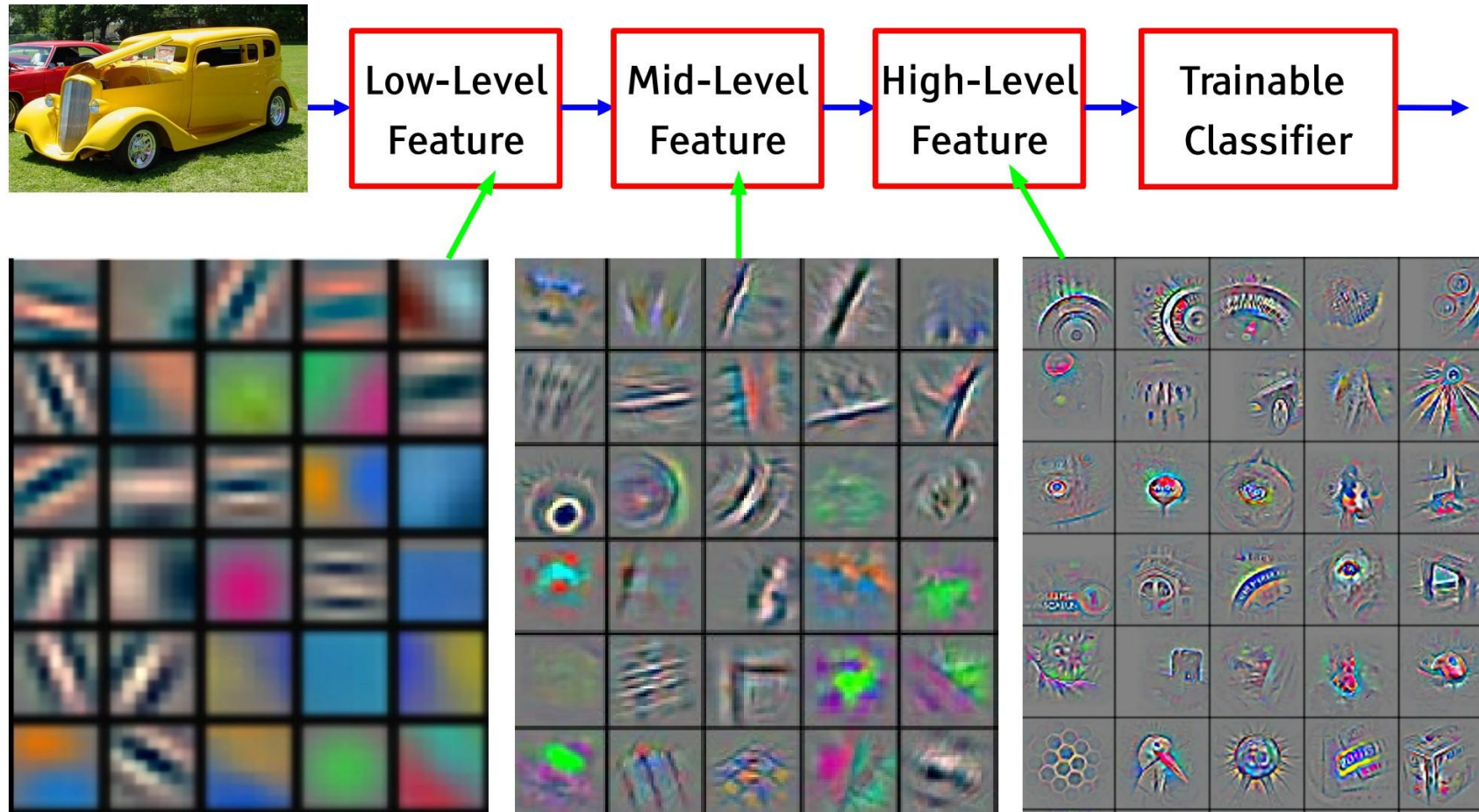
First conv layer: kernel 11x11x3x96 stride 4

Kernel shape: (11,11,3,96)

Output shape: (55,55,96)

Number of parameters: 34,944

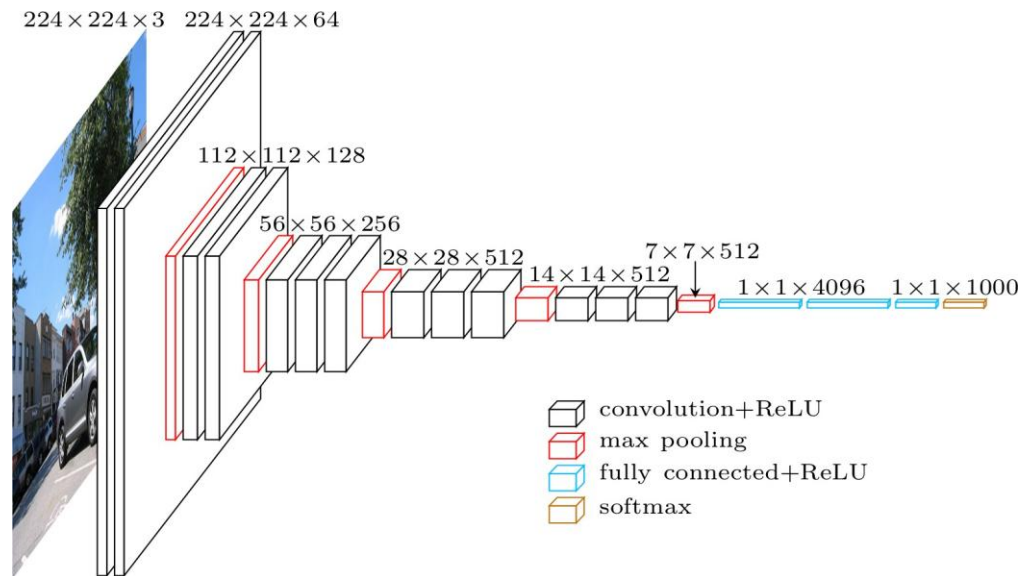
Hierarchical Representation



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

VGG-16

Deep convolutional network that **consists of 16 layers** with **3×3 convolutional filters**, designed for feature extraction while maintaining spatial hierarchies.



Very deep convolutional networks for large-scale image recognition. 2014.

```
model.add(Convolution2D(64, 3, 3, activation='relu', input_shape=(3, 224, 224)))
model.add(Convolution2D(64, 3, 3, activation='relu'))
model.add(MaxPooling2D((2, 2), strides=(2, 2)))
```

```
model.add(Convolution2D(128, 3, 3, activation='relu'))
model.add(Convolution2D(128, 3, 3, activation='relu'))
model.add(MaxPooling2D((2, 2), strides=(2, 2)))
```

```
model.add(Convolution2D(256, 3, 3, activation='relu'))
model.add(Convolution2D(256, 3, 3, activation='relu'))
model.add(Convolution2D(256, 3, 3, activation='relu'))
model.add(MaxPooling2D((2, 2), strides=(2, 2)))
```

```
model.add(Convolution2D(512, 3, 3, activation='relu'))
model.add(Convolution2D(512, 3, 3, activation='relu'))
model.add(Convolution2D(512, 3, 3, activation='relu'))
model.add(MaxPooling2D((2, 2), strides=(2, 2)))
```

```
model.add(Convolution2D(512, 3, 3, activation='relu'))
model.add(Convolution2D(512, 3, 3, activation='relu'))
model.add(Convolution2D(512, 3, 3, activation='relu'))
model.add(MaxPooling2D((2, 2), strides=(2, 2)))
```

```
model.add(Flatten()) model.add(Dense(4096,
activation='relu')) model.add(Dropout(0.5))
model.add(Dense(4096, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(1000, activation='softmax'))
```

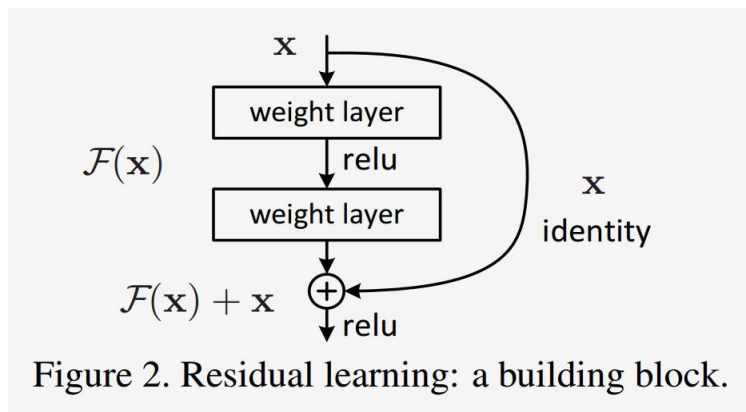
ResNet

A deep neural network that **introduces skip (or residual) connections** to help training **deeper architectures** by mitigating the vanishing gradient problem. They allow gradients to flow through layers more easily, enabling the training of extremely deep models.

Even deeper models:

34, 50, 101, 152 layers

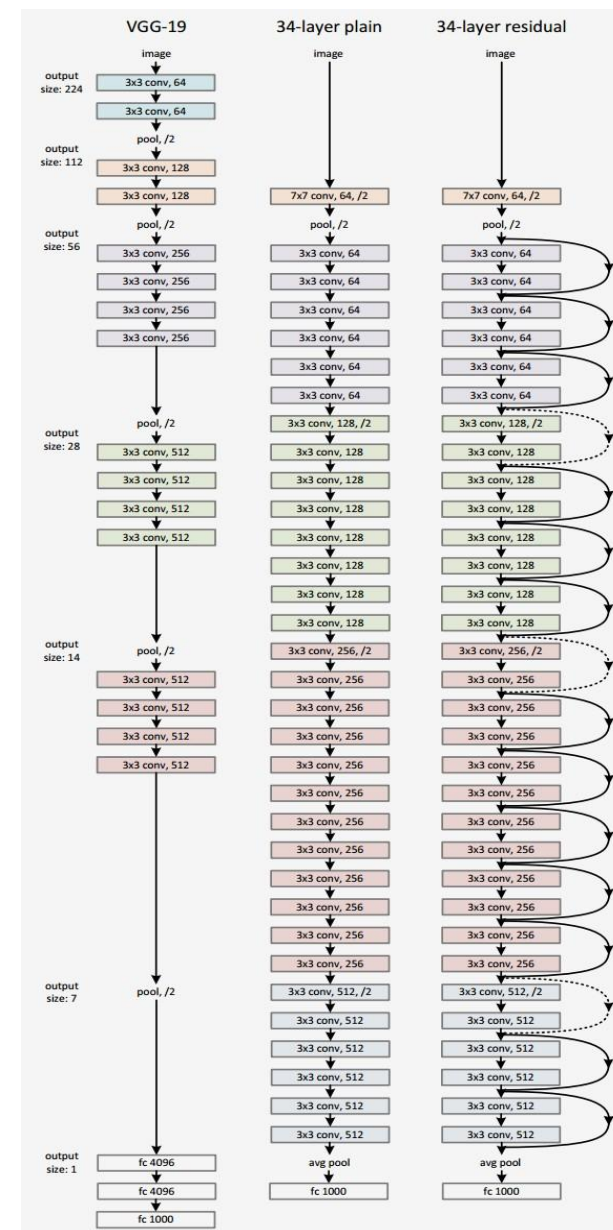
A block learns the **residual** w.r.t. identity



Good optimization properties!

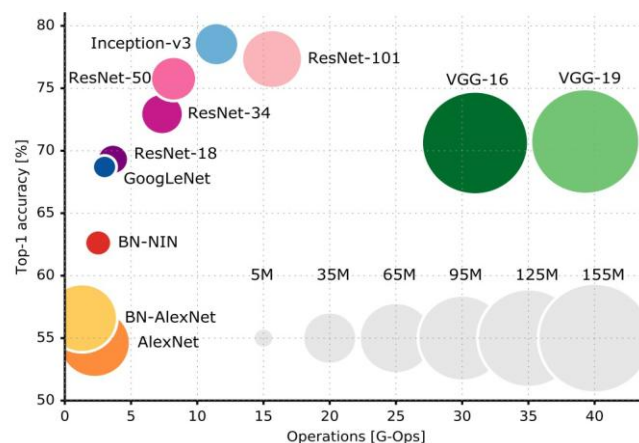
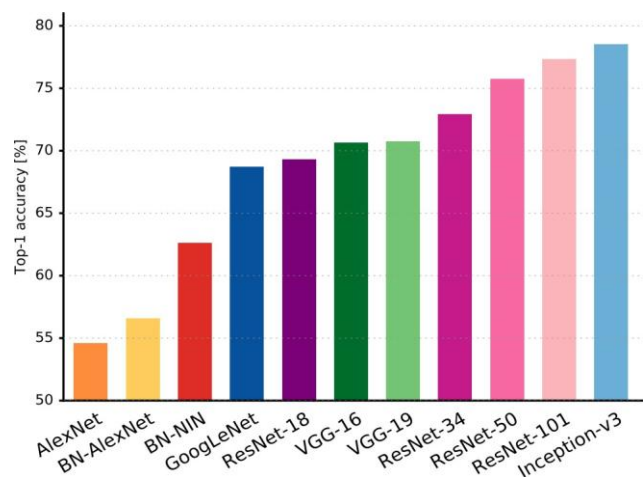
ResNet50 vs. VGG

- Superior accuracy in all vision tasks
5.25% top-5 error vs 7.1%
- Less parameters
25M vs 138M
- Computational complexity
3.8B Flops vs 15.3B Flops
- Fully Convolutional until the last layer

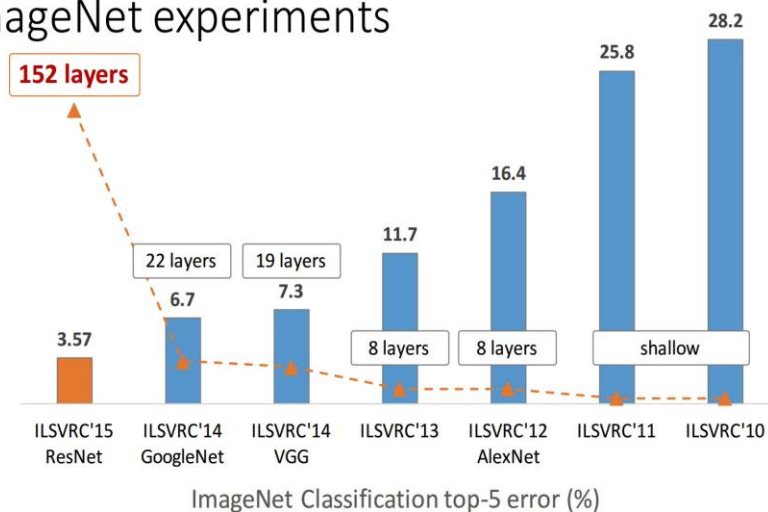


Deep residual learning for image recognition, CVPR. 2016.

State-Of-The-Art



ImageNet experiments



Method	# Params	Extra Data	ImageNet	
			Top-1	Top-5
ResNet-50 [24]	26M	—	76.0	93.0
ResNet-152 [24]	60M	—	77.8	93.8
DenseNet-264 [28]	34M	—	77.9	93.9
Inception-v3 [62]	24M	—	78.8	94.4
Xception [11]	23M	—	79.0	94.5
Inception-v4 [61]	48M	—	80.0	95.0
Inception-resnet-v2 [61]	56M	—	80.1	95.1
ResNeXt-101 [78]	84M	—	80.9	95.6
PolyNet [87]	92M	—	81.3	95.8
SENet [27]	146M	—	82.7	96.2
NASNet-A [90]	89M	—	82.7	96.2
AmoebaNet-A [52]	87M	—	82.8	96.1
PNASNet [39]	86M	—	82.9	96.2
AmoebaNet-C + AutoAugment [12]	155M	—	83.5	96.5
GPipe [29]	557M	—	84.3	97.0
EfficientNet-B7 [63]	66M	—	85.0	97.2
EfficientNet-B7 + FixRes [70]	66M	—	85.3	97.4
EfficientNet-L2 [63]	480M	—	85.5	97.5
ResNet-50 Billion-scale SSL [79]	26M	3.5B labeled Instagram	81.2	96.0
ResNeXt-101 Billion-scale SSL [79]	193M	3.5B labeled Instagram	84.8	—
ResNeXt-101 WSL [42]	829M	3.5B labeled Instagram	85.4	97.6
FixRes ResNeXt-101 WSL [69]	829M	3.5B labeled Instagram	86.4	98.0
Big Transfer (BiT-L) [33]	928M	300M labeled JFT	87.5	98.5
Noisy Student (EfficientNet-L2) [77]	480M	300M unlabeled JFT	88.4	98.7
Noisy Student + FixRes [70]	480M	300M unlabeled JFT	88.5	98.7
Vision Transformer (ViT-H) [14]	632M	300M labeled JFT	88.55	—
EfficientNet-L2-NoisyStudent + SAM [16]	480M	300M unlabeled JFT	88.6	98.6
Meta Pseudo Labels (EfficientNet-B6-Wide)	390M	300M unlabeled JFT	90.0	98.7
Meta Pseudo Labels (EfficientNet-L2)	480M	300M unlabeled JFT	90.2	98.8

<https://paperswithcode.com/sota/image-classification-on-imagenet>

Pre-Trained Models

Training a model on ImageNet from scratch takes days or weeks.

Many models trained on ImageNet and their weights **are publicly available!**

Transfer Learning:

- Use pre-trained weights, remove last layers to compute representations of images.
- Train a classification model from these features on a new classification task.
- The network is used as a generic feature extractor.
- Better than handcrafted feature extraction on natural images.

Fine Tuning:

Retraining the (some) parameters of the network (given enough data).

- Truncate the last layer(s) of the pre-trained network.
- Freeze the remaining layers weights.
- Add a (linear) classifier on top and train it for a few epochs.
- Then fine-tune the whole network or the few deepest layers.
- Use a smaller learning rate when fine tuning.

Data Augmentation

- Data augmentation is a technique to **artificially expand datasets by applying transformations**.
- It helps **improve machine learning model generalization and reduces overfitting**.
- Widely used in computer vision, natural language processing, and audio processing.

Why Use Data Augmentation?

- Increases dataset size without collecting new data.
- Helps models generalize better to unseen data.
- Reduces the risk of overfitting by introducing variability.
- Improves model robustness against real-world variations.



Augmented versions from original images

Transformations applied such as rotation, flipping, brightness adjustment.

Types of Data Augmentation

Images

1. **Geometric Transformations:** rotation, flipping, cropping, translation.
2. **Color-Based Augmentations:** brightness, contrast, saturation changes.
3. **Noise Injection:** adding blur, Gaussian noise, sharpening.
4. **Advanced Methods:** MixUp, CutMix, random erasing.

Text and Audio

1. **Text Augmentation:** synonym replacement, back-translation, sentence shuffling.
2. **Audio Augmentation:** time stretching, pitch shifting, noise injection.
3. **Augmentation in NLP:** word embedding perturbations, paraphrasing.

Convolutional Neural Networks

Working Lab



Examples To Play Around...

Examples to refresh how to play around with convolutional neural networks:

- Binary CNN (*binary_cnn* Jupyter notebook)
 - MNIST dataset
- Multiclass Colour CNN (*multiclass_colour_cnn* Jupyter notebook)
 - CIFAR-10 dataset

Introduction to Recurrent Neural Networks



What is Natural Language Processing (NLP)?

Definition

- “An interdisciplinary subfield of linguistics, computer science, and artificial intelligence concerned with the interactions between computers and human language, in particular how to program computers to process and analyze large amounts of natural language data.” (Wikipedia)
- NLP = NLU + NLG
 - Natural Language Understanding (NLU)
 - Natural Language Generation (NLG)

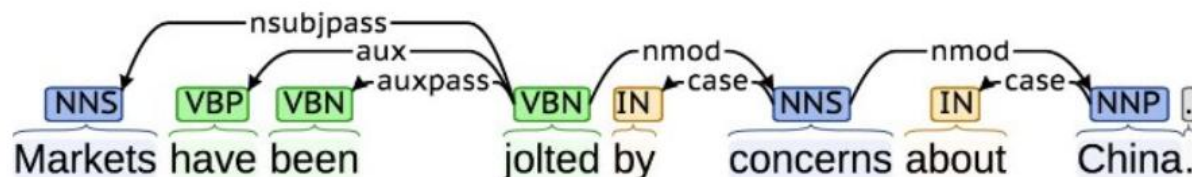
Apple **ORG** is looking at buying U.K. **GPE** startup for \$1 billion **MONEY**

TEXT	START	END	LABEL	DESCRIPTION
Apple	0	5	ORG	Companies, agencies, institutions.
U.K.	27	31	GPE	Geopolitical entity, i.e. countries, cities, states.
\$1 billion	44	54	MONEY	Monetary values, including unit.

Named Entity Recognition

Sentence	Negative	Neutral	Positive
This is just terrible. I will not buy it.	89.70%	08.04%	02.26%
I may buy this one, but I'm not sure.	15.54%	50.21%	34.25%
Amazing product!	00.10%	00.76%	99.14%
My friend loves this product. I will sure buy it!	00.39%	01.17%	98.44%

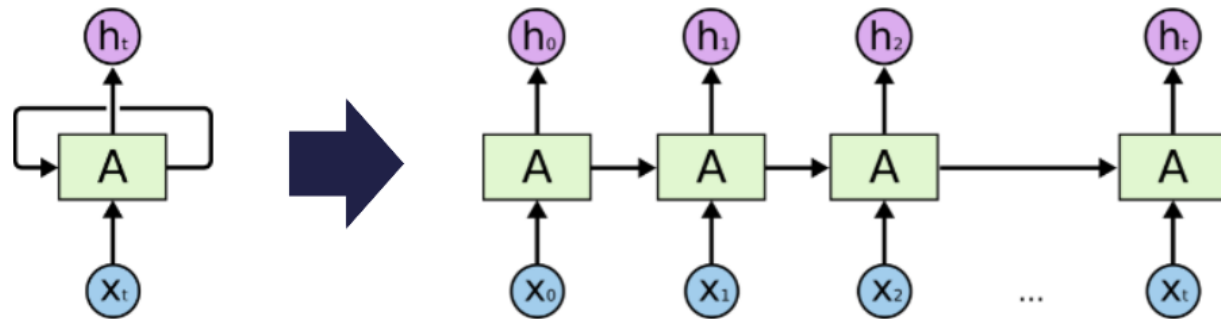
Sentiment analysis



Dependency parsing for sentence structure

Recurrent Neural Network (RNN)

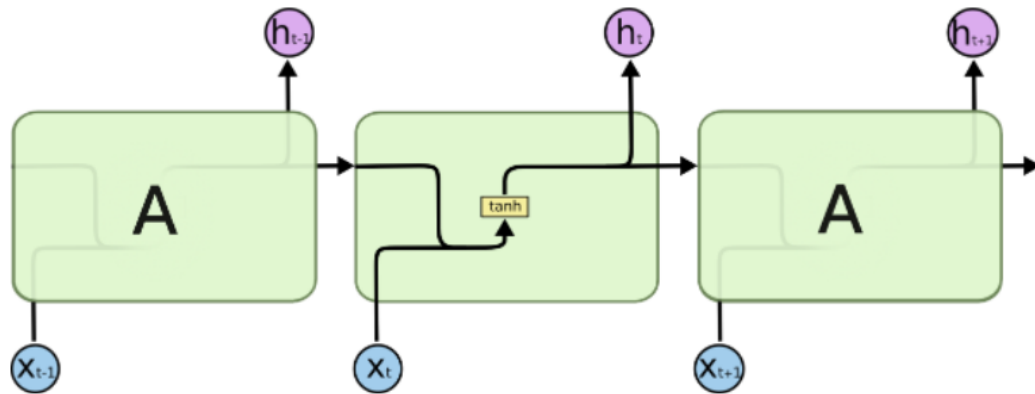
Quick overview in the context of language modeling



The problem of long-term dependencies

“I grew up in **Brazil** in a city named New Hamburg, and I speak fluent ”

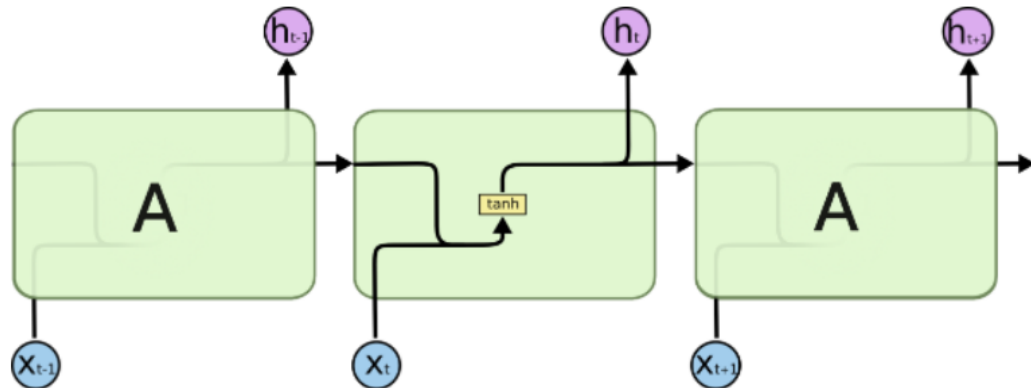
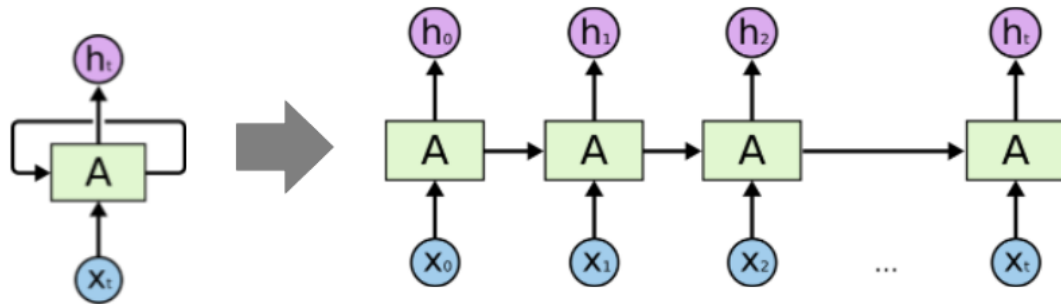
$p(\text{sandcastle} \mid \text{Alice went to the beach. There she built a})$



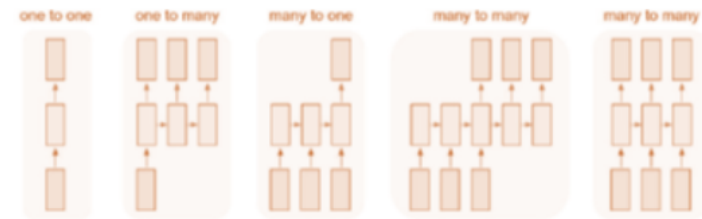
Some challenges/issues

- Challenges to dealing with long-term dependencies
- Vanishing gradient or/and exploding gradient
- Slow training process

Key Milestones



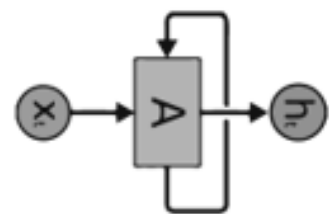
- **Recurrent Neural Networks (RNN)** are networks **with loops**, allowing **information to persist**. They map from the entire history of previous inputs to each output to handle sequential data.
- RNNs can be thought of as **multiple copies of the same network**, each **passing a message to a successor**. The same function and the same set of params are used at every time step.
- Back-Propagation Through Time.
- The model is represented as a multi-layer one (with an unbounded num. of layers) and backpropagation is applied on the unrolled model.



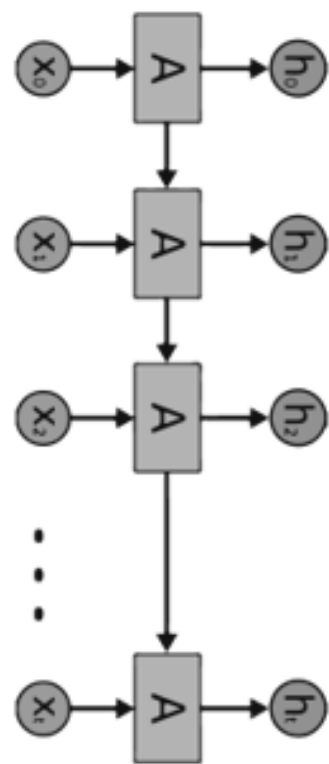
RNN could experience these **issues**: 1) challenges to **dealing with long-term dependencies**, 2) **exploding gradient** (clip the gradients to a certain max value), 3) **vanishing gradient** (introducing memory via LSTM or GRU), and 4) **slow training process**.

Why transformers?

- **Sequences in science:** spectrums, chemical chains, hadron jet structures, measurement readings ...
 - Either: symbolic representation like protein structures
 - Or: continuous values like experimental measurements
- **State of the art:** RNNs, LSTMs...
 - Have been around since 90s
 - Intuitive to understand once you wrap your head around time-notation
 - Are good for reasonably short sequences
 - However, not easily to parallel for long sequence training, and building bigger and bigger models is cumbersome (intractable) due to gradient vanishing



||



Key Milestones: The Transformer

The **Transformer** is a novel architecture that aims to solve **sequence-to-sequence** tasks while **handling long-range dependencies**. The **core** idea of the Transformer model is the **attention mechanism**, which weighs the importance of different words when producing an output word.

- **Encoder:** each layer has a multi-head self-attention + fully connected neural network
- **Decoder:** each layer has a masked multi-head self attention + multi-head self-attention + fully connected neural network

Motivation:

- Reduce computation complexity per layer
- Make computation parallelizable
- Be capable of learning long-range dependencies

